Example 5:

create a docker image

- name as **ubuntujava** with tag as **v1**
- take ubuntu as the base image,
- update package manager & install java (or default-jre)
- Whenever we start a container **echo hello world** command to get executed

```
FROM ubuntu
RUN apt update -y
RUN apt install default-jre -y
CMD echo "hello world"
```

observation: build dockerfile → create image → create container

once container gets started observe the CMD instruction gets executed (echo command)

What is COPY in Docker?

COPY instruction is used for copying the files from the docker host into to the containers file system (or image).

syntax: **COPY <files/Directory_name_in_dockerhost> <path_inside_container>**

What is ADD in Docker?

ADD instruction is also used for copying files from docker host into the container filesystem (or image).

ADD can also be used for downloading files from remote servers.

syntax: **ADD <files/Directory_name_in_dockerhost> <path_inside_container>**

example of ADD to download file from url:

ADD `https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.42/bin/apache-tomcat-10.1.42-fulldocs.tar.gz` **/opt**

Note: Above add instruction will download tomcat 10.1.42, unarchives it & copies it into image to /opt/ directory

Example 6 (usage of COPY):
Create a dockerfile by taking
- alpine as the base image
- copy samplefile to docker container to /opt directory
- CMD instruction to list file in /opt

Construct an image from the dockerfile.

 # Before building make sure you have created a file called samplefile in DockerHost

```
FROM alpine
# Copy samplefile from host to /opt in container
COPY samplefile /opt/
CMD ["ls", "/opt"] #this same is ls /opt
```

Observation: build dockerfile → create image → create container → container will list files in /opt, check if samplefile is present or not

Example 7:

Create a dockerfile by taking
- alpine as the base image
- download maven installation file to docker container /opt directory

Construct an image from the dockerfile.

```
FROM alpine
# Use ADD to download and extract Maven archive to /opt
ADD https://dlcdn.apache.org/maven/maven-3/3.9.10/binaries/apache-maven-3.9.10-bin.zip /opt/
```

Observation: build dockerfile → create image → create container → inside container → verify maven tar.gz download

Note:

- Alpine & busybox are light weight (smaller sized) docker images which will have all linux utilities
- Each instruction in Dockerfile is called as Layers
- Assume your custom Dockerfile is named MyDockerfile (instead of Dockerfile) and is in the current directory.

```
docker build -f MyDockerfile -t my-custom-image .
```

What is difference between COPY & ADD instruction in Dockerfile?

Both COPY & ADD instructions are used to copy files/Directories from Docker Host to Container. ADD can also be used to download file (like wget command in linux) & also ADD can untar/unzip file inside containers,

Feature	COPY	ADD
Copy a local file or directory from your host into Docker image	YES	YES
Download a file from a URL into Docker image	NO	YES
Extract a tar file from source to Docker image	NO	YES

ENV

ENV instruction is used to set environment variables inside the container.

These variables can be accessed by the application running in the container.

. syntax: **`ENV <Variable_name> <variable_value>`**

example: **`ENV sportsman viratkohli`**

build dockerfile → create image → create container → inside container → echo $sportsman

EXPOSE

Is used to document the port on which application would run in container

Its only for documentation purpose, while creating container we have to use port mapping.

Example 8:

Create a dockerfile by taking

- base image ubuntu
- create a user as devopsuser & make him as default user after logging in container
- set default working directory as opt

Construct an image from the dockerfile.

vi Dockerfile

```
FROM ubuntu
RUN usedadd devopsuser
USER devopsuser
WORKDIR /opt
```

observation: build dockerfile → create image → create container → inside container verify default user & default directory

****IMPORTANT****

How Long Does a Docker Container Run?

- Every docker image is built to run a **main process** (defined by the CMD or ENTRYPOINT).
- As long as **main process** is running, the container will be running condition.

Once the **main process** execution is completed, the container will get itself moved to exited / stopped state.

(main process --> whatever mentioned in CMD instruction in Dockerfile)

How Can I Check What Process My Container is Running?

in already running containers, we can check default process using docker ps -qa command & observe under COMMAND sections

Practicals & observations on understanding default process in containers:

scenario 1:

Create Dockerfile with below mentioned instructions

```
FROM ubuntu
CMD ["date"]
```

build this Dockerfile & create a conatiner from it & observe that conatiner has moved in to exited state.

Reason?
- container has exited, because it has completed running its default process (whatever mentioned in CMD instruction i.e date command)

Note:

For all linux based containers (ubuntu, centos, amazonlinux), the default process is shell process for that Dockerfile will look like

```
FROM ubuntu
CMD ["/bin/bash"]
```

/bin/bash or bash -- is nothing but the terminal.
Hence we are able to enter -it mode in ubuntu/centos or any OS based containers.

What is an Image Registry (or Container Registry)?

image registry is a centralized location for storing and distributing docker images.
we can **push (upload)** our images to the registry or **pull (download)** them whenever needed

Types of Registries

1. Public Registry

- These are open to everyone across the globe.
- Anyone can **download (pull)** images without login.
  Example: **Docker Hub**

2. Private Registry

- Access is restricted to certain users or teams (within a company).
- Secure and suitable for enterprise use.
  Examples:
  - **JFrog Artifactory**
  - **Sonatype Nexus**
  - **AWS ECR** (Elastic Container Registry)
  - **Azure ACR** (Azure Container Registry)

Create a DockerHub Account

1. Visit: https://hub.docker.com/
2. Click on Sign Up
3. Fill in details like Docker ID, email, and password
4. Confirm your email and log in

To upload the image to hub.docker.com (docker login command is used)

$ docker login -u <> -p <> (provide dockerId and dockerhub password)

Once authenticated, you'll see: `login Succeeded`

Note: Pushing image to dockerhub:

if you're pushing an image to **DockerHub**, the image name **must include your DockerHub username** (also called Docker ID) as a prefix.

For Example:

```
docker build -t <DockerID>/<image-name>:<tag> .
```

docker build -t *shwetha11/myapp:v1*

```
docker push < DockerID>/<imag-name>:<tag>
```

docker push *shwetha11/myapp:v1*

Create a docker image named amazonlinuxgit with tag as 2023, take amazonlinux as the base image, git to be available inside the container, Construct an image from the dockerfile & push it to docker hub.

vi Dockerfile

```
FROM amazonlinux
RUN yum update -y
RUN yum install git -y
```

docker build -t <dockerHub_userID>/<imagename>:<tag> .

docker push <dockerHub_userID>/<imagename>:<tag>

login to docker hub GUI to see your image

Note on steps to deploy acecloudacademy web application:

step 1. launch ec2 instances amazonlinux (security group inbound rules)
step 2. installing pre-requisite Softwares git , java , maven
step 3. clone acecloudacademy repo & cd in to acecloudacademy directory
step 4. Build the source code (mvn clean package)
--> artifacts will get generated (war file) <--
step 5. tomcat instance ==> copy the war file --> webapps

Assignment:

1. Generate artifacts war file from above steps
   - Write a Docker file with tomee as a base image
   - COPY generated war file to webapps directory inside the container & expose on port==> EXPOSE 8080
   - build Dockerfile to create image & run container so that we can access ===> acecloudacademy webapplication

2. How to copy files from dockerHost to already running container
3. How to rename docker image name(re tagging)
4. What is the Docker Socket File (/var/run/docker.sock)?

Differences between CMD & ENTRYPOINT instructions in Dockerfile

CMD → we will define the commands that needs to be executed when a container starts.
ENTRYPOINT → we will define the executable that needs to be used when a container starts.

Note on CMD:

- CMD sets the default command that needs to be executed when a container starts.
- CMD command/instruction can be easily overridden while creating a container with different command.

Note on ENTRYPOINT:

- ENTRYPOINT command/instruction can't be overridden while creating a container.
- Most of Docker containers by default have ENTRYPOINT ==> which is ==> /bin/sh -c
- ENTRYPOINT generally used if we want to container as an executable (like only purpose of running container is to run a script only)

Difference between CMD & ENTRYPOINT		
Feature	**CMD**	**ENTRYPOINT**
Purpose	Sets the default command to run	Configures a container to run as an executable (Like script)
Overriding	Can be overridden at runtime	Cannot be overridden by default
Use Case	When you want to provide default parameters for commands	When you want to ensure a specific command runs regardless of the parameters
Flexibility	More flexible; easily overridden	More rigid; ensures the main process is always executed
Use Together	Often used with ENTRYPOINT for defaults	Can include CMD arguments as options to ENTRYPOINT

Below practical examples demonstrating the difference between CMD and ENTRYPOINT in a Dockerfile:

Example 1: Using CMD alone Dockerfile:

```
FROM ubuntu
CMD ["echo", "Hello, World!"]
```

Build the Dockerfile to get image:
$ docker `build -t myimage1 .`

On Running the container:

```
$ docker run my-image1
Hello, World!
```

```
$ docker run my-image1  echo "welcome"
Welcome
```

Observations: In this case, the CMD instruction sets the default command to echo "Hello, World!". However, the user can override this command when starting the container, as shown in the second example where the container runs echo "welcome" instead.

Example 2: Using ENTRYPOINT alone - Dockerfile

```
FROM ubuntu
ENTRYPOINT ["echo"]
```

Build the Dockerfile to get image:

```
$ docker build -t myimage2 .
```

On Running the container:
Case 1)

```
$ docker run my-image2 "Hello, World!"
Hello, World!
```

Case 2)

```
$ docker run my-image2
Usage: echo [OPTION]... [STRING]…
```

Observations: In this case, the ENTRYPOINT instruction sets the executable that will be the main process in the container. The user can provide arguments to this executable when starting the container, as shown in the first example. However, the user cannot override the ENTRYPOINT command itself.

Example 3: Using CMD and ENTRYPOINT together Dockerfile:

```
FROM ubuntu
ENTRYPOINT ["echo"]
CMD ["Hello, World!"]
```

Build the Dockerfile to get image:

```
$ docker build -t myimage3 .
```

On Running the container:

```
$ docker run my-image3
Hello, World!
```

```
$ docker run my-image3 "Goodbye, World!"
Goodbye, World!
```

Observations: In this case, the ENTRYPOINT sets the executable to echo, and the CMD provides the default arguments to be passed to the ENTRYPOINT command. The user can override the arguments by providing their own when starting the container, as shown in the second example.

These examples demonstrate how CMD and ENTRYPOINT work together to provide flexibility in defining the main process and its arguments in a Docker container.

Conclusion:

- CMD sets default commands and arguments but can be overridden.
- ENTRYPOINT makes the container behave like an executable and has a higher priority over CMD.